

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEPyN



Manual para el
alumno

Kubernetes II



WE WANT
SKILLS!

mundosE

Manual para el Alumno: Kubernetes II.....	2
Objetivos del encuentro:.....	2
1. Escalado automático con HPA.....	3
2. Gestión avanzada de actualizaciones: RollingUpdate y Rollback.....	5
3. Gestión avanzada con YAML: ConfigMaps, Secrets y almacenamiento persistente.....	7
4. Introducción a Helm.....	9
Bibliografía.....	11

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontraremos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Kubernetes II

Objetivos del encuentro:

- Explicar el concepto de **escalado automático** en Kubernetes y el rol del **Horizontal Pod Autoscaler (HPA)**.
- Describir **estrategias de actualización** que reducen el riesgo (RollingUpdate, Recreate) y cómo funcionan **rollbacks** y revisiones.
- Diferenciar y aplicar conceptualmente **ConfigMaps, Secrets** y **almacenamiento persistente** (PV/PVC/StorageClass), entendiendo sus implicancias de seguridad y gobernanza.
- Comprender el valor de **Helm** como sistema de plantillas/versionado para gestionar despliegues repetibles y auditables.

Contenido a desarrollar:

- ✓ Escalado automático con Horizontal Pod Autoscaler (HPA)
- ✓ Gestión avanzada de actualizaciones (RollingUpdate y Rollback)
- ✓ Gestión avanzada con YAML: ConfigMaps, Secrets y almacenamiento persistente (PV/PVC/StorageClass)
- ✓ Introducción a Helm: Charts, values y releases

1. Escalado automático con HPA

Kubernetes propone un modelo declarativo donde defines el **estado deseado** y el sistema se encarga de alcanzarlo de manera continua.

El **Horizontal Pod Autoscaler (HPA)** es uno de los mecanismos que materializa esa idea: ajusta la **cantidad de réplicas** de una carga de trabajo en función de **métricas observadas**. El objetivo es lograr **elasticidad** (adaptarse a picos y valles de demanda), mantener **desempeño** estable frente a variaciones de carga y optimizar **costos** evitando sobreaprovisionamiento.

Desde una mirada conceptual, el HPA se comporta como un **controlador de realimentación**. Observa una métrica (por ejemplo, utilización media de CPU por Pod), la compara con un **objetivo** y actúa para disminuir la diferencia: si la métrica supera el umbral, aumenta réplicas; si queda por debajo, reduce. Para evitar variaciones bruscas, incorpora **ventanas de promediado** y **periodos de estabilización** que filtran el ruido y amortiguan reacciones excesivas. Esta noción de control subraya que el HPA no es “mágico”: su efectividad depende de **señales adecuadas** y **parámetros** coherentes con el comportamiento de la aplicación.

El HPA consume tres familias de métricas:

- (a) **recursos** nativos (CPU y memoria),
- (b) **métricas de objeto** (p. ej., tasa interna de peticiones de un recurso Kubernetes), y
- (c) **métricas personalizadas/externas** como latencia p95, **profundidad de cola**, tasa de errores, QPS o cualquier indicador relevante para el servicio.

Elegir la familia correcta es una **decisión de diseño**. En cargas **CPU-bound**, la utilización de CPU suele correlacionar con demanda. En sistemas **I/O-bound** o asincrónicos, métricas como cola pendiente o latencia describen mejor la presión real. Lo importante es que la **métrica de control** refleje el **SLO** (objetivo de nivel de servicio): si buscas latencia < X ms, conviene que el autoscaler observe directamente ese fenómeno —o un proxy fiable— y no una señal débilmente correlacionada.

Para métricas de recursos, Kubernetes calcula la utilización **respecto del request** configurado en el contenedor. Esto implica que **definir correctamente requests y limits** es un prerequisite: sin **request** no existe base para interpretar “80 % de uso”. Además, **requests** influyen en el **scheduler** (reserva de CPU/memoria) y, por lo tanto, en la **densidad** y la **colocación** de Pods. Un diseño sano alinea **requests** con consumo típico y deja margen razonable en **limits** para picos breves, evitando tanto el estrangulamiento por *throttling* como el desperdicio.

Otra pieza clave es la **granularidad temporal**. Ventanas de observación demasiado cortas pueden reaccionar a *spikes* efímeros (**flapping**); ventanas largas demoran la respuesta ante cambios reales. El **tiempo de arranque** de la aplicación también pesa: si una nueva réplica tarda en calentar *caches*, conectarse a dependencias y quedar *ready*, el beneficio del escalado tardará en materializarse. Aquí la **readiness** funciona como compuerta: hasta que un Pod no está listo, no debe contarse como capacidad efectiva.

El HPA interactúa con otras capas: el **autoscaler de clúster** (que agrega o quita **nodos**) y el **Vertical Pod Autoscaler (VPA)** (que ajusta *requests* y *limits*). Conceptualmente, conviene **coordinar** estas piezas: si el HPA pide réplicas que no entran en los nodos actuales, el autoscaler del clúster debería poder ampliar capacidad; si el VPA aumenta *requests* al mismo tiempo que el HPA sube réplicas, el sistema puede volverse errático. Una práctica teórica común es: HPA activo sobre **réplicas** y VPA en modo **recomendación** o aplicado a cargas distintas para evitar conflictos.

Como toda estrategia, el HPA tiene **limitaciones**: presupone trabajo **divisible** (si tu servicio no paraleliza, más réplicas no mejoran *throughput*) y puede verse frenado por **cuellos de botella compartidos** (bases de datos, colas, *caches*). También hay efectos colaterales como **sesiones pegajosas** o **calentamiento de caché por instancia** que pueden degradar los beneficios del escalado. Por eso, el diseño debe considerar la **topología completa** del sistema y no sólo la capa de aplicación.

En materia de **gobernanza**, es recomendable instrumentar **observabilidad** que permita auditar al HPA: ¿cuándo escala?, ¿cuánto tarda?, ¿logra estabilizar los SLO?, ¿se detectan oscilaciones? Definir **réplicas mínimas** (piso de resiliencia) y **máximas** (techo de costo) supone analizar demanda histórica, estacionalidad y ventanas de tráfico. A nivel conceptual, el HPA es exitoso cuando opera como **traducción automática** de objetivos de negocio (respuesta consistente con uso eficiente de recursos) en decisiones de **capacidad** casi en tiempo real.

Recursos para profundizar:

The Kubernetes Authors © (2025):

- *Horizontal Pod Autoscaler*:
<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- *Autoscaling Workloads*.
<https://kubernetes.io/docs/concepts/workloads/autoscaling/>
- *Resource Management for Pods and Containers*.
<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

2. Gestión avanzada de actualizaciones: RollingUpdate y Rollback

Las actualizaciones son inevitables: corregir *bugs*, introducir funciones, mejorar seguridad o *performance*. El desafío es **cambiar sin romper**. Kubernetes ofrece mecanismos para **minimizar el impacto**, **recuperarse** rápido si algo falla y **auditar** lo ocurrido.

La abstracción **Deployment** concentra estos conceptos con una estrategia por defecto: **RollingUpdate**.

Teóricamente, *RollingUpdate* funciona como un **flujo gradual** que reemplaza instancias antiguas por nuevas de forma escalonada. Esta gradualidad acota el **radio de blast** y permite validar que las nuevas réplicas alcanzan estado *ready* antes de retirar las anteriores.

Dos parámetros definen la “forma” del *rollout*: **maxSurge** (capacidad adicional temporal permitida) y **maxUnavailable** (indisponibilidad tolerada). Puedes expresarlos como número absoluto o porcentaje respecto del total de réplicas.

- Valores **conservadores** privilegian continuidad (menos indisponibilidad a costa de mayor duración);
- valores **agresivos** reducen el tiempo total pero aumentan la exposición.

Otro ajuste conceptual es **minReadySeconds**, que impide considerar “disponible” a una réplica hasta que permanezca lista por cierto tiempo, evitando falsos positivos de *readiness* durante el *warm-up*. La **readiness** en sí misma actúa como llave: el sistema solo enruta tráfico hacia Pods listos.

El **progreso** se evalúa con un umbral temporal (**progressDeadlineSeconds**). Si la nueva versión no logra condiciones de disponibilidad en ese plazo, el Deployment se marca como **no progresando**, habilitando acciones correctivas, entre ellas el **rollback**. Detrás de escena, los Deployments conservan un **historial** de *ReplicaSets* (revisiones), de modo que **volver atrás** es restaurar un estado deseado anterior y re-promover el RS correspondiente. Desde la teoría de gobernanza, esto traduce el principio de **reversibilidad del cambio**.

Durante un RollingUpdate conviven réplicas de versiones distintas porque los **Services** enrutan por **labels** y **readiness**, no por versión. Esa coexistencia exige **compatibilidad hacia atrás** del contrato (API, mensajes, esquema de datos). Sin compatibilidad, incluso un *rollout* perfecto puede quebrar el sistema. En este punto aparecen **patrones** de ingeniería de liberaciones: **migraciones en dos**

pasos (primero *expand*: aceptar ambos esquemas; luego *contract*: retirar lo viejo), **feature flags** (activar gradualmente), y **adapters** o *shims* que aíslan cambios de interfaz.

Además de RollingUpdate, existe **Recreate**, que detiene primero la versión vieja y luego levanta la nueva. Conceptualmente es simple pero implica **ventana de indisponibilidad**; solo tiene sentido cuando no es posible la convivencia (por ejemplo, cambios incompatibles irreconciliables a nivel de datos) o cuando la aplicación no tolera múltiples versiones simultáneas por restricciones de estado compartido.

Patrones de mayor alcance suman control adicional sobre el **enrutamiento de tráfico**. En **canary**, una fracción pequeña (p. ej., 5%) apunta a la versión nueva; si **métricas objetivos** (errores, latencia, saturación) se mantienen dentro de los umbrales, la proporción se incrementa. El valor teórico radica en **detectar temprano** degradaciones. En **blue/green**, se prepara un entorno paralelo (*green*); cuando las verificaciones son satisfactorias, se conmuta el tráfico desde *blue* hacia *green*. La ventaja conceptual es un **rollback instantáneo** (volver a *blue*); el costo es mayor consumo de recursos durante la preparación.

La **seguridad operativa** de un despliegue incluye **Pod Disruption Budgets (PDB)** para limitar indisponibilidad por intervenciones voluntarias, **terminación elegante** para cerrar conexiones y tareas en curso, y **probes** de salud bien diseñadas para que el sistema no avance con réplicas defectuosas. También importa la **observabilidad**: métricas, logs y *tracing* permiten evaluar si la nueva versión cumple los SLO definidos.

En cargas **stateful**, la abstracción **StatefulSet** implementa actualizaciones **ordenadas** (una por vez) y permite **particionar** el *rollout* (detenerse en un ordinal) cuando se requiere coordinación con *quorum* o réplicas líderes/seguidoras. **DaemonSet**, en cambio, actualiza **una réplica por nodo** preservando la cobertura de agentes del sistema.

En síntesis, la gestión avanzada de actualizaciones en Kubernetes combina **mecanismos de gradualidad, reversión y contratos estables**. La teoría indica que el éxito no depende solo de "cambiar la imagen" sino de alinear estrategia, compatibilidad y observabilidad, con umbrales y políticas claras que definan cuándo avanzar, pausar o revertir.

Recursos para profundizar:

The Kubernetes Authors © (2025):

- Deployments – estrategia *RollingUpdate* y progreso.
<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

- Probes (liveness/readiness/startup):
<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes>
- Introducción a patrones *canary* y *blue/green*:
<https://kubernetes.io/docs/concepts/cluster-administration/manage-deployment/>

3. Gestión avanzada con YAML: ConfigMaps, Secrets y almacenamiento persistente

En Kubernetes, **YAML** es el idioma del estado deseado. Más allá de Pods y Deployments, un dominio clave es el de la **configuración externa** y el **estado persistente**. Aquí entran **ConfigMaps**, **Secrets** y el trío **PV/PVC/StorageClass**. Entender su rol te permite separar **código** de **configuración**, proteger **material sensible** y desacoplar la aplicación de la infraestructura de almacenamiento.

ConfigMap es un contenedor de pares clave-valor orientado a datos **no sensibles**. Su aporte central es habilitar que un mismo artefacto (la imagen) funcione en múltiples entornos con parámetros diferentes sin reconstruir. Conceptualmente, esto promueve **inmutabilidad** y **promoción** entre ambientes. Para que sea efectivo, la aplicación debe adoptar principios de **configuración externa** (leer desde variables de entorno o archivos montados) y, si corresponde, soportar **recarga** de configuración.

Secret comparte el modelo clave-valor pero para **información sensible** (credenciales, tokens, certificados). A nivel teórico, el hecho de que los datos estén codificados en base-64 **no** aporta seguridad por sí mismo; la protección real proviene de:

- (1) **controles de acceso** (RBAC),
- (2) **cifrado en reposo** del almacenamiento del clúster,
- (3) prácticas de **rotación** y **alcance mínimo**, y
- (4) decisiones de **montaje** (permisos, *securityContext*, usuarios no privilegiados).

Separar secretos de configuración no sensible facilita auditar quién accede a qué y aplicar el principio de **menor privilegio**.

Al actualizar un ConfigMap o Secret, el efecto en los Pods depende de **cómo** lo consumen. Si se montó como **archivo**, algunos controladores refrescan el contenido sin reiniciar; si se inyectó como **variable de entorno**, los procesos conservarán el valor cargado al inicio. Este matiz impulsa patrones como

checksum en anotaciones para forzar recreación de Pods cuando cambia la configuración, y evita inconsistencias entre réplicas.

Para **almacenamiento persistente**, Kubernetes separa **solicitud** y **provisión**.

- Un **PersistentVolumeClaim (PVC)** declara **capacidad, modo de acceso** y, opcionalmente, **StorageClass**.
- Un **PersistentVolume (PV)** satisface esa solicitud, ya sea de forma **estática** (pre-creado) o **dinámica** (creado por un **provisionador** cuando aparece el PVC).
- Las **StorageClass** encapsulan parámetros del backend: tipo de disco, IOPS, zona, replicación, *binding mode* y política de *reclaim* (qué sucede con el volumen cuando se libera el claim: **Retain, Delete** o **Recycle**).

Entender los **modos de acceso** —**ReadWriteOnce, ReadWriteMany, ReadOnlyMany**— ayuda a definir si el volumen será de **escritura exclusiva** o **multi-escritor**, algo crucial para bases de datos y sistemas distribuidos.

La **forma del volumen** también importa: **filesystem** vs. **block**. En modo *block*, la aplicación recibe un dispositivo sin sistema de archivos y lo maneja directamente; algunos motores de datos prefieren este enfoque para optimizar E/S. Finalmente, existen volúmenes **efímeros** respaldados por controladores CSI que nacen y mueren con el Pod, útiles para *scratch* o cachés que requieren mayor control que `emptyDir`.

Desde la **gobernanza**, conviene aplicar **políticas de admisión y validación**: límites de tamaño para Secrets, etiquetas obligatorias, convenciones de nombres, cuotas por *namespace*, y perfiles de **seguridad de Pods** (niveles *baseline/restricted*). Complementariamente, **NetworkPolicy** acota la comunicación entre Pods y protege datos en tránsito dentro del clúster. Estas políticas no son “decoración”: constituyen el **contrato organizacional** que garantiza consistencia y seguridad a lo largo del tiempo.

En términos de **mantenibilidad**, organizar los YAML en módulos, documentar decisiones en **annotations**, evitar anidamientos innecesarios y usar **nombres coherentes** reduce el costo cognitivo. Anclas y referencias pueden ahorrar repetición, pero su abuso dificulta la lectura. Si se usan herramientas de plantillas (Helm, Kustomize), es recomendable conservar el **manifiesto renderizado** como evidencia auditable de lo que realmente llega al clúster.

Recursos para profundizar:

The Kubernetes Authors © (2025):

- ConfigMaps y Secrets (conceptos).
<https://kubernetes.io/docs/concepts/configuration/>
- Almacenamiento persistente (PV/PVC/StorageClass).

<https://kubernetes.io/docs/concepts/storage/>

- Políticas y seguridad de Pods.

<https://kubernetes.io/docs/concepts/security/pod-security-standards/>

4. Introducción a Helm

Helm se suele presentar como el “gestor de paquetes” de Kubernetes, pero su utilidad verdadera es más amplia: ofrece un **sistema de plantillas** y **versionado de despliegues** que permite empaquetar configuraciones repetibles y auditables. Un **chart** agrupa recursos (Deployments, Services, Ingress, ConfigMaps, etc.), metadatos (nombre, versión, dependencias) y **plantillas** parametrizables. Los **values** representan la configuración concreta aplicada en una instalación, y una **release** es el resultado instalado en el clúster con un nombre y un **historial** de revisiones.

El valor conceptual de Helm reside en **separar** lo **genérico** (plantillas) de lo **específico** (valores por entorno). Esto permite que un mismo *chart* se **especialice** para desarrollo, prueba y producción simplemente cambiando archivos de valores, sin duplicar manifiestos. En términos de gobernanza, cada actualización de una release genera una **revisión**, facilitando **rollback** cuando un cambio no cumple expectativas. Esta trazabilidad formaliza el **control de cambios** y se alinea con prácticas de auditoría.

Helm utiliza **Go templates** y funciones **Sprig** para componer manifiestos a partir de valores. Este *templating* habilita **abstracciones reutilizables**: *helpers* para nombres, etiquetas estándares, afinidades, toleraciones o reglas de *ingress*. No obstante, incorporar demasiada lógica en las plantillas aumenta la **complejidad cognitiva** y dificulta el razonamiento. La recomendación teórica es mantener plantillas **declarativas y legibles**, reservando la variabilidad a **values** bien documentados y coherentes.

La **gestión de dependencias** se aborda mediante **subcharts**. Un chart principal puede declarar componentes auxiliares (por ejemplo, base de datos, cola de mensajes, *ingress controller*), favoreciendo la **composición**. Aquí cobra relevancia el **versionado semántico (SemVer)**: el campo **version** refleja cambios en el **chart** (plantillas y estructura), mientras **appVersion** comunica la versión de la **aplicación** empaquetada. Diferenciar ambas dimensiones ayuda a fijar **políticas de compatibilidad** y a prever el impacto de los cambios.

Otra idea clave es la **precedencia de valores**. Helm define un orden que resuelve conflictos entre valores **predeterminados** del chart y valores **específicos** aportados en la instalación/actualización. Comprender esa jerarquía evita sorpresas y facilita diseñar **valores por entorno** sin duplicación. En organizaciones grandes, es común establecer **convenciones** (nombres, etiquetas) y **valores**

globales para asegurar uniformidad.

En materia de **seguridad**, es preferible **no almacenar secretos en texto plano** dentro de *values*. En su lugar, se recomiendan integraciones con gestores externos o patrones de **secretos sellados**. Helm además soporta **firmado y verificación** de *charts*, reforzando la **integridad de la cadena de suministro**: verificar procedencia e integridad antes de instalar reduce el riesgo de introducir artefactos manipulados. Los **hooks** de ciclo de vida (pre/post install/upgrade/delete) permiten orquestar tareas auxiliares —por ejemplo, migraciones de esquema o validaciones automáticas— siempre que se usen con **medida**, evitando ocultar complejidad esencial detrás de automatismos opacos.

Finalmente, Helm se integra naturalmente con enfoques **declarativos** como **GitOps**, donde *charts* y *values* viven en repositorios versionados y un reconciliador aplica el estado deseado al clúster. Esta combinación aporta **trazabilidad, reversibilidad y reproducibilidad** a los despliegues. En suma, Helm no solo acelera instalaciones repetitivas: establece un **marco de gobernanza** para el cambio continuo, con separación clara entre plantillas y valores, y con controles sobre procedencia y evolución de los artefactos.

Recursos para profundizar:

Helm Authors © (2025):

- Documentación oficial de Helm (conceptos, *charts*, seguridad): <https://helm.sh/docs/>
- *Charts* y best practices: <https://helm.sh/docs/topics/charts/>
- Firma y verificación de *charts*: <https://helm.sh/docs/topics/provenance/>

Bibliografía

- The Kubernetes Authors. *Kubernetes Documentation* – secciones Concepts, Workloads, Autoscaling, Storage. <https://kubernetes.io/docs/>
- Burns, B.; Beda, J.; y, Hightower, K. (2022). *Kubernetes: Up & Running*. 3.^a ed., O'Reilly.
- Burns, J.; Grant, D. *Kubernetes Patterns* (2.^a ed.). O'Reilly, 2023.
- Tao, W.; Hüttermann, M. *GitOps and Kubernetes*. Apress, 2022.
- Helm Authors. *Helm Documentation*. <https://helm.sh/docs/>
- CNCF. *Security & Policy* (glosario y prácticas). <https://www.cncf.io/projects/>